MODULE END ASSIGNMENT

E-COMMERCE CHURN ANALYSIS

PREPARED BY:

AMRA ABDUL HAMEED

Problem Statement:

In the realm of e-commerce, businesses face the challenge of understanding customer churn patterns to ensure customer satisfaction and sustained profitability. This project aims to delve into the dynamics of customer churn within an e-commerce domain, utilizing historical transactional data to uncover underlying patterns and drivers of churn. By analyzing customer attributes such as tenure, preferred payment modes, satisfaction scores, and purchase behavior, the project seeks to investigate and understand the dynamics of customer attrition and their propensity to churn. The ultimate objective is to equip e-commerce enterprises with actionable insights to implement targeted retention strategies and mitigate churn, thereby fostering long-term customer relationships and ensuring business viability in a competitive landscape.

DATABASE:

```
DROP DATABASE IF EXISTS ecomm;
CREATE DATABASE ecomm;
USE ecomm;

CREATE TABLE customer_churn(
   CustomerID                INT  PRIMARY KEY,
   Churn                     BIT,
   Tenure                    INT,
   PreferredLoginDevice      VARCHAR(20),
   CityTier                  INT,
   WarehouseToHome           INT,
   PreferredPaymentMode      VARCHAR(20),
   Gender                    ENUM('Male','Female'),
   HourSpendOnApp            INT,
   NumberOfDeviceRegistered  INT,
   PreferedOrderCat          VARCHAR(20),
   SatisfactionScore         INT,
   MaritalStatus             VARCHAR(10),
   NumberOfAddress           INT,
   Complain                  BIT,
   OrderAmountHikeFromlastYear INT,
   CouponUsed                INT,
   OrderCount                INT,
   DaySinceLastOrder         INT,
   CashbackAmount            INT
   );
```

And then 5630 rows of data is added.

```
(55630,0,8,'Mobile Phone',1,15,'Credit Card','Male',3,2,'Laptop & Accessory',3,'Married',4,0,13,2,2,3,169);

SELECT * FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpendOnApp	NumberOfDeviceRegistered	PreferedC
50001	1	4	Mobile Phone	3	6	Debit Card	Female	3	3	Laptop & A
50002	1	NULL	Phone	1	8	UPI	Male	3	4	Mobile
50003	1	NULL	Phone	1	30	Debit Card	Male	2	4	Mobile
50004	1	0	Phone	3	15	Debit Card	Male	2	4	Laptop & A
50005	1	0	Phone	1	12	CC	Male	NULL	3	Mobile
50006	1	0	Computer	1	22	Debit Card	Female	3	5	Mobile Pho
50007	1	NULL	Phone	3	11	Cash on Delivery	Male	2	3	Laptop & A

Result Grid | Form Editor

```
SELECT COUNT(*) AS Total_Customers
FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_Customers
5630

DATA CLEANING:

1. Handling Missing Values and Outliers:

➢ Impute mean for the following columns, and round off to the nearest integer if required: WarehouseToHome, HourSpendOnApp, OrderAmountHikeFromlastYear, DaySinceLastOrder.

➢ Impute mode for the following columns: Tenure, CouponUsed, OrderCount.

➢ Handle outliers in the 'WarehouseToHome' column by deleting rows where the values are greater than 100.

First lets find all the missing values.

```sql
SELECT
    COUNT(*) AS Total_Rows,
    SUM(WarehouseToHome IS NULL) AS Missing_WarehouseToHome,
    SUM(HourSpendOnApp IS NULL) AS Missing_HourSpendOnApp,
    SUM(OrderAmountHikeFromlastYear IS NULL) AS Missing_OrderAmountHike,
    SUM(DaySinceLastOrder IS NULL) AS Missing_DaySinceLastOrder,
    SUM(Tenure IS NULL) AS Missing_Tenure,
    SUM(CouponUsed IS NULL) AS Missing_CouponUsed,
    SUM(OrderCount IS NULL) AS Missing_OrderCount
FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_Rows	Missing_WarehouseToHome	Missing_HourSpendOnApp	Missing_OrderAmountHike	Missing_DaySinceLastOrder	Missing_Tenure	Missing_CouponUsed	Missing_OrderCou
5628	251	255	265	307	264	256	258

Now, Remove WarehouseToHome outliers

```sql
SELECT *
FROM customer_churn
WHERE WarehouseToHome > 100;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpendOnApp	NumberOfDeviceRegistered	PreferedOrde
51310	0	25	Computer	3	126	Debit Card	Male	2	3	Mobile
54125	0	26	Computer	3	127	Debit Card	Male	3	4	Mobile Phone
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Form Editor

```sql
DELETE FROM customer_churn
WHERE WarehouseToHome > 100;
```

Output will be

3 22 19:47:26	DELETE FROM customer_churn WHERE WarehouseToHome > 100	2 row(s) affected	0.031 sec

The following columns need mean imputation:

- `WarehouseToHome`
- `HourSpendOnApp`
- `OrderAmountHikeFromlastYear`
- `DaySinceLastOrder`

```sql
SELECT
    ROUND(AVG(WarehouseToHome)) AS Avg_WarehouseToHome,
    ROUND(AVG(HourSpendOnApp)) AS Avg_HourSpendOnApp,
    ROUND(AVG(OrderAmountHikeFromlastYear)) AS Avg_OrderAmountHike,
    ROUND(AVG(DaySinceLastOrder)) AS Avg_DaySinceLastOrder
FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Avg_WarehouseToHome	Avg_HourSpendOnApp	Avg_OrderAmountHike	Avg_DaySinceLastOrder
16	3	16	5

Now update the missing values:

```sql
UPDATE customer_churn
SET WarehouseToHome = (
    SELECT avg_value
    FROM (
        SELECT ROUND(AVG(WarehouseToHome)) AS avg_value
        FROM customer_churn
    ) AS temp
)
WHERE WarehouseToHome IS NULL;
```

tput

Action Output

#	Time	Action	Messag
23	19:47:32	SET SQL_SAFE_UPDATES = 1	0 row(s)
24	19:50:38	SELECT COUNT(*) AS Total_Rows, SUM(WarehouseToHome IS NULL) AS Missing_WarehouseToHom...	1 row(s)
25	19:54:17	SELECT ROUND(AVG(WarehouseToHome)) AS Avg_WarehouseToHome, ROUND(AVG(HourSpendOn...	1 row(s)
26	19:55:01	UPDATE customer_churn SET WarehouseToHome = (SELECT avg_value FROM (SELECT ROU...	Error Co
27	19:55:12	SET SQL_SAFE_UPDATES = 0	0 row(s)
28	19:55:18	UPDATE customer_churn SET WarehouseToHome = (SELECT avg_value FROM (SELECT ROU...	251 row

```sql
UPDATE customer_churn
SET HourSpendOnApp = (
    SELECT avg_value
    FROM (
        SELECT ROUND(AVG(HourSpendOnApp)) AS avg_value
        FROM customer_churn
    ) AS temp
)
WHERE HourSpendOnApp IS NULL;
```

```sql
UPDATE customer_churn
SET OrderAmountHikeFromlastYear = (
    SELECT avg_value
    FROM (
        SELECT ROUND(AVG(OrderAmountHikeFromlastYear)) AS avg_value
        FROM customer_churn
    ) AS temp
)
WHERE OrderAmountHikeFromlastYear IS NULL;
```

```sql
UPDATE customer_churn
SET DaySinceLastOrder = (
    SELECT avg_value
    FROM (
        SELECT ROUND(AVG(DaySinceLastOrder)) AS avg_value
        FROM customer_churn
    ) AS temp
)
WHERE DaySinceLastOrder IS NULL;
```

Now lets do mode imputation on the following columns.

- `Tenure`

```sql
SELECT Tenure, COUNT(*) AS Frequency
FROM customer_churn
WHERE Tenure IS NOT NULL
GROUP BY Tenure
ORDER BY Frequency DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

Tenure	Frequency
1	690

- CouponUsed

```sql
SELECT CouponUsed, COUNT(*) AS Frequency
FROM customer_churn
WHERE CouponUsed IS NOT NULL
GROUP BY CouponUsed
ORDER BY Frequency DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap C

CouponUsed	Frequency
1	2104

- OrderCount

```sql
SELECT OrderCount, COUNT(*) AS Frequency
FROM customer_churn
WHERE OrderCount IS NOT NULL
GROUP BY OrderCount
ORDER BY Frequency DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderCount	Frequency
2	2024

For this dataset, the modes are:

- **Tenure = 1**
- **CouponUsed = 1**
- **OrderCount = 2**

Now lets update each one .

```
UPDATE customer_churn
SET Tenure = 1
WHERE Tenure IS NULL;
```

Output

```
UPDATE customer_churn
SET CouponUsed = 1
WHERE CouponUsed IS NULL;
```

```
UPDATE customer_churn
SET OrderCount = 2
WHERE OrderCount IS NULL;
```

Now lets check the missing values.

```
SELECT
    SUM(Tenure IS NULL) AS Missing_Tenure,
    SUM(WarehouseToHome IS NULL) AS Missing_WarehouseToHome,
    SUM(HourSpendOnApp IS NULL) AS Missing_HourSpendOnApp,
    SUM(OrderAmountHikeFromlastYear IS NULL) AS Missing_OrderAmountHike,
    SUM(CouponUsed IS NULL) AS Missing_CouponUsed,
    SUM(OrderCount IS NULL) AS Missing_OrderCount,
    SUM(DaySinceLastOrder IS NULL) AS Missing_DaySinceLastOrder
FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Missing_Tenure	Missing_WarehouseToHome	Missing_HourSpendOnApp	Missing_OrderAmountHike	Missing_CouponUsed	Missing_OrderCount	Missing_DaySinceLastOrder
0	0	0	0	0	0	0

All done.

Dealing with Inconsistencies:

➢ Replace occurrences of “Phone” in the 'PreferredLoginDevice' column and

“Mobile” in the 'PreferedOrderCat' column with “Mobile Phone” to ensure uniformity.

➢ Standardize payment mode values: Replace "COD" with "Cash on Delivery" and "CC" with "Credit Card" in the PreferredPaymentMode column.

Preferred login device is set to mobile phone from phone here,

```
113 •  UPDATE customer_churn
114    SET PreferredLoginDevice = 'Mobile Phone'
115    WHERE PreferredLoginDevice = 'Phone';
```

Output

Action Output

#	Time	Action	Message
34	19:59:31	SELECT OrderCount, COUNT(*) AS Frequency FROM customer_churn WHERE OrderCount IS NOT NULL G...	1 row(s) returned
35	20:00:12	UPDATE customer_churn SET Tenure = 1 WHERE Tenure IS NULL	264 row(s) affected
36	20:00:57	UPDATE customer_churn SET CouponUsed = 1 WHERE CouponUsed IS NULL	256 row(s) affected
37	20:01:25	UPDATE customer_churn SET OrderCount = 2 WHERE OrderCount IS NULL	258 row(s) affected
38	20:02:14	SELECT SUM(Tenure IS NULL) AS Missing_Tenure, SUM(WarehouseToHome IS NULL) AS Missing_W...	1 row(s) returned
39	20:03:28	UPDATE customer_churn SET PreferredLoginDevice = 'Mobile Phone' WHERE PreferredLoginDevice = 'Phone'	1231 row(s) affecte

Preferred login device is set to mobile phone from mobile here,

```
17 •  UPDATE customer_churn
18    SET PreferedOrderCat = 'Mobile Phone'
19    WHERE PreferedOrderCat = 'Mobile';
```

tput

Prefered payment mode is set to cash on delivery from cod;

```
121 •  UPDATE customer_churn
122    SET PreferredPaymentMode = 'Cash on Delivery'
123    WHERE PreferredPaymentMode = 'COD';
```

Prefered payment mode is set to credit card from cc;

```sql
UPDATE customer_churn
SET PreferredPaymentMode = 'Credit Card'
WHERE PreferredPaymentMode = 'CC';
```

Now lets check.

```sql
SELECT PreferredPaymentMode, COUNT(*) AS Customer_Count
FROM customer_churn
GROUP BY PreferredPaymentMode;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredPaymentMode	Customer_Count
Debit Card	2312
UPI	414
Credit Card	1774
Cash on Delivery	514
E wallet	614

Data Transformation:

Column Renaming:

➢ Rename the column "PreferedOrderCat" to "PreferredOrderCat".

➢ Rename the column "HourSpendOnApp" to "HoursSpentOnApp".

```sql
USE ecomm;

ALTER TABLE customer_churn
RENAME COLUMN PreferedOrderCat TO PreferredOrderCat;

ALTER TABLE customer_churn
RENAME COLUMN HourSpendOnApp TO HoursSpentOnApp;
```

Creating New Columns:

➢ Create a new column named ‘ComplaintReceived’ with values "Yes" if the corresponding value in the ‘Complain’ is 1, and "No" otherwise.

```sql
ALTER TABLE customer_churn
ADD COLUMN ComplaintReceived VARCHAR(3);

UPDATE customer_churn
SET ComplaintReceived =
    CASE
        WHEN Complain = 1 THEN 'Yes'
        ELSE 'No'
    END;
```

```sql
SELECT ComplaintReceived, COUNT(*) AS Customer_Count
FROM customer_churn
GROUP BY ComplaintReceived;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ComplaintReceived	Customer_Count
Yes	1604
No	4024

➢ Create a new column named 'ChurnStatus'. Set its value to “Churned” if the corresponding value in the 'Churn' column is 1, else assign “Active”.

```sql
ALTER TABLE customer_churn
ADD COLUMN ChurnStatus VARCHAR(10);

UPDATE customer_churn
SET ChurnStatus =
    CASE
        WHEN Churn = 1 THEN 'Churned'
        ELSE 'Active'
    END;
```

```sql
SELECT ChurnStatus, COUNT(*) AS Customer_Count
FROM customer_churn
GROUP BY ChurnStatus;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ChurnStatus	Customer_Count
Churned	948
Active	4680

Column Dropping:

➢ Drop the columns "Churn" and "Complain" from the table.

```sql

ALTER TABLE customer_churn
DROP COLUMN Churn,
DROP COLUMN Complain;
```

Output

Data Exploration and Analysis:

➢ Retrieve the count of churned and active customers from the dataset.

```sql
SELECT
    ChurnStatus,
    COUNT(*) AS Customer_Count
FROM customer_churn
GROUP BY ChurnStatus;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ChurnStatus	Customer_Count
Churned	948
Active	4680

➢ Display the average tenure and total cashback amount of customers who

churned.

```sql
SELECT
    ROUND(AVG(Tenure), 2) AS Average_Tenure,
    SUM(CashbackAmount) AS Total_Cashback
FROM customer_churn
WHERE ChurnStatus = 'Churned';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Average_Tenure	Total_Cashback
3.18	152030

➢ Determine the percentage of churned customers who complained.

```sql
SELECT
    ROUND(
        100.0 * SUM(ComplaintReceived = 'Yes') / COUNT(*),
        2
    ) AS Complaint_Percentage
FROM customer_churn
WHERE ChurnStatus = 'Churned';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Complaint_Percentage
53.59

➢ Identify the city tier with the highest number of churned customers whose preferred order category is Laptop & Accessory.

```sql
SELECT
    CityTier,
    COUNT(*) AS Customer_Count
FROM customer_churn
WHERE ChurnStatus = 'Churned'
  AND PreferredOrderCat = 'Laptop & Accessory'
GROUP BY CityTier
ORDER BY Customer_Count DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CityTier	Customer_Count
3	150
1	60

➢ Identify the most preferred payment mode among active customers.

```sql
SELECT
    PreferredPaymentMode,
    COUNT(*) AS Customer_Count
FROM customer_churn
WHERE ChurnStatus = 'Active'
GROUP BY PreferredPaymentMode
ORDER BY Customer_Count DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

PreferredPaymentMode	Customer_Count
Debit Card	1956
Credit Card	1522
E wallet	474
Cash on Delivery	386
UPI	342

➢ Calculate the total order amount hike from last year for customers who are single and prefer mobile phones for ordering.

```sql
SELECT
    SUM(OrderAmountHikeFromlastYear) AS Total_OrderAmount_Hike
FROM customer_churn
WHERE MaritalStatus = 'Single'
  AND PreferredOrderCat = 'Mobile Phone';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Total_OrderAmount_Hike
▶	12177

➢ Find the average number of devices registered among customers who used UPI as their preferred payment mode.

```sql
SELECT
  ROUND(AVG(NumberOfDeviceRegistered), 2) AS Average_Devices
FROM customer_churn
WHERE PreferredPaymentMode = 'UPI';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Average_Devices
▶	3.72

➢ Determine the city tier with the highest number of customers.

```sql
SELECT
    CityTier,
    COUNT(*) AS Customer_Count
FROM customer_churn
GROUP BY CityTier
ORDER BY Customer_Count DESC;
```

Result Grid | Filter Rows: | Export: | W

CityTier	Customer_Count
1	3666
3	1720
2	242

➢ Identify the gender that utilized the highest number of coupons.

```sql
SELECT
    Gender,
    SUM(CouponUsed) AS Total_Coupons
FROM customer_churn
GROUP BY Gender
ORDER BY Total_Coupons DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Gender	Total_Coupons
Male	5629
Female	4034

➢ List the number of customers and the maximum hours spent on the app in each preferred order category.

```sql
SELECT
    PreferredOrderCat,
    COUNT(*) AS Customer_Count,
    MAX(HoursSpentOnApp) AS Maximum_Hours_Spent
FROM customer_churn
GROUP BY PreferredOrderCat
ORDER BY PreferredOrderCat;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredOrderCat	Customer_Count	Maximum_Hours_Spent
Fashion	826	5
Grocery	410	4
Laptop & Accessory	2050	5
Mobile Phone	2078	5
Others	264	4

➢ Calculate the total order count for customers who prefer using credit cards and have the maximum satisfaction score.

```sql
SELECT
    SUM(OrderCount) AS Total_Order_Count
FROM customer_churn
WHERE PreferredPaymentMode = 'Credit Card'
  AND SatisfactionScore = (
      SELECT MAX(SatisfactionScore)
      FROM customer_churn
  );
```

Result Grid | Filter Rows: | Export: | Wrap Cell C

Total_Order_Count
1122

➢ What is the average satisfaction score of customers who have complained?

```sql
SELECT
    ROUND(AVG(SatisfactionScore), 2) AS Average_Satisfaction_Score
FROM customer_churn
WHERE ComplaintReceived = 'Yes';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Average_Satisfaction_Score
3.00

➢ List the preferred order category among customers who used more than 5 coupons.

```sql

SELECT
    PreferredOrderCat,
    COUNT(*) AS Customer_Count
FROM customer_churn
WHERE CouponUsed > 5
GROUP BY PreferredOrderCat
ORDER BY Customer_Count DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

PreferredOrderCat	Customer_Count
Laptop & Accessory	99
Fashion	89
Mobile Phone	45
Grocery	42
Others	28

Result 13 ×

➢ List the top 3 preferred order categories with the highest average cashback amount.

```sql
SELECT
    PreferredOrderCat,
    ROUND(AVG(CashbackAmount), 2) AS Average_Cashback
FROM customer_churn
GROUP BY PreferredOrderCat
ORDER BY Average_Cashback DESC
LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

PreferredOrderCat	Average_Cashback
Others	304.45
Grocery	266.24
Fashion	210.40

➢ Find the preferred payment modes of customers whose average tenure is 10 months and have placed more than 500 orders.

```sql
SELECT
    PreferredPaymentMode,
    Tenure,
    OrderCount
FROM customer_churn
WHERE Tenure = 10
  AND OrderCount > 500;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredPaymentMode	Tenure	OrderCount

It returns no records. Because the dataset's `OrderCount` does not reach 500.

➢ Categorize customers based on their distance from the warehouse to home such as 'Very Close Distance' for distances <=5km, 'Close Distance' for <=10km, 'Moderate Distance' for <=15km, and 'Far Distance' for >15km. Then, display the churn status breakdown for each distance category.

```sql
ALTER TABLE customer_churn
ADD COLUMN DistanceCategory VARCHAR(30);
UPDATE customer_churn
SET DistanceCategory =
    CASE
        WHEN WarehouseToHome <= 5 THEN 'Very Close Distance'
        WHEN WarehouseToHome <= 10 THEN 'Close Distance'
        WHEN WarehouseToHome <= 15 THEN 'Moderate Distance'
        ELSE 'Far Distance'
    END;
```

```sql
SELECT
    DistanceCategory,
    ChurnStatus,
    COUNT(*) AS Customer_Count
FROM customer_churn
GROUP BY DistanceCategory, ChurnStatus
ORDER BY
    CASE DistanceCategory
        WHEN 'Very Close Distance' THEN 1
        WHEN 'Close Distance' THEN 2
        WHEN 'Moderate Distance' THEN 3
        WHEN 'Far Distance' THEN 4
    END,
    ChurnStatus;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DistanceCategory	ChurnStatus	Customer_Count
Very Close Distance	Active	7
Very Close Distance	Churned	1
Close Distance	Active	1696
Close Distance	Churned	265
Moderate Distance	Active	1106
Moderate Distance	Churned	184
Far Distance	Active	1871
Far Distance	Churned	498

Result 16

Output

➢ List the customer's order details who are married, live in City Tier-1, and their order counts are more than the average number of orders placed by all

customers.

The overall average order count is :

```
SELECT
    AVG(OrderCount) AS Average_Order_Count
FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Average_Order_Count
2.9623

```
SELECT *
FROM customer_churn
WHERE MaritalStatus = 'Married'
  AND CityTier = 1
  AND OrderCount > (
      SELECT AVG(OrderCount)
      FROM customer_churn
  );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HoursSpentOnApp	NumberOfDeviceRegistered	PreferredOrderCat
50049	3	Computer	1	15	Credit Card	Male	3	3	Laptop & Accessory
50119	13	Mobile Phone	1	30	Cash on Delivery	Male	3	3	Laptop & Accessory
50132	15	Mobile Phone	1	16	Credit Card	Female	2	4	Mobile Phone
50133	13	Mobile Phone	1	8	Credit Card	Male	2	3	Others
50152	2	Mobile Phone	1	28	Debit Card	Female	3	2	Fashion
50180	12	Mobile Phone	1	6	Credit Card	Male	2	2	Fashion
50187	18	Computer	1	13	Debit Card	Male	3	3	Fashion

customer_churn 18 | Apply

Output

Action Output

There are 576 records that satisfy all three conditions.

➢ a) Create a ‘customer_returns’ table in the ‘ecomm’ database and insert the following data:

ReturnID CustomerID ReturnDate RefundAmount
1001 50022 2023-01-01 2130
1002 50316 2023-01-23 2000
1003 51099 2023-02-14 2290
1004 52321 2023-03-08 2510
1005 52928 2023-03-20 3000
1006 53749 2023-04-17 1740
1007 54206 2023-04-21 3250
1008 54838 2023-04-30 1990

```sql
CREATE TABLE customer_returns (
  ReturnID INT PRIMARY KEY,
  CustomerID INT,
  ReturnDate DATE,
  RefundAmount INT,
  FOREIGN KEY (CustomerID)
      REFERENCES customer_churn(CustomerID)
);
```

```sql
INSERT INTO customer_returns
(ReturnID, CustomerID, ReturnDate, RefundAmount)
VALUES
(1001, 50022, '2023-01-01', 2130),
(1002, 50316, '2023-01-23', 2000),
(1003, 51099, '2023-02-14', 2290),
(1004, 52321, '2023-03-08', 2510),
(1005, 52928, '2023-03-20', 3000),
(1006, 53749, '2023-04-17', 1740),
(1007, 54206, '2023-04-21', 3250),
(1008, 54838, '2023-04-30', 1990);
```

b) Display the return details along with the customer details of those who have churned and have made complaints.

```sql
SELECT
    r.ReturnID,
    r.CustomerID,
    r.ReturnDate,
    r.RefundAmount,
    c.ChurnStatus,
    c.ComplaintReceived,
    c.Tenure,
    c.PreferredLoginDevice,
    c.CityTier,
    c.WarehouseToHome,
    c.PreferredPaymentMode,
    c.Gender,
    c.HoursSpentOnApp,
    c.NumberOfDeviceRegistered,
    c.PreferredOrderCat,
    c.SatisfactionScore,
    c.MaritalStatus,
    c.NumberOfAddress,
    c.OrderAmountHikeFromlastYear,
    c.CouponUsed,
    c.OrderCount,
    c.DaySinceLastOrder,
    c.CashbackAmount
FROM customer_returns r
JOIN customer_churn c
    ON r.CustomerID = c.CustomerID
WHERE c.ChurnStatus = 'Churned'
  AND c.ComplaintReceived = 'Yes';
```

Output

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ReturnID	CustomerID	ReturnDate	RefundAmount	ChurnStatus	ComplaintReceived	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	G
1002	50316	2023-01-23	2000	Churned	Yes	0	Computer	2	29	UPI	Fe
1004	52321	2023-03-08	2510	Churned	Yes	18	Mobile Phone	3	19	E wallet	Ma
1006	53749	2023-04-17	1740	Churned	Yes	1	Mobile Phone	3	31	Credit Card	Fe

Result Grid

Form Editor